

Data Structures and Algorithms: Basics, Complexity, and ADTs

Gruppe #1— Hand-in #1

7. september 2026

Navn	stud.nr.
Luka Simonsen	201509438
Frederik Gammelgaard Nielsen	202508129
Dean Holgaard Jensen	202004340
Alexander Tønder	202309616

1. Number of integers in a vector

- (a) (Week 1) Write a program that generates random M integers and puts them in an vector. Then it generates another N random integers and counts how many of them are in the array using an iterator.

Solution:

```
int main() {
    std::random_device rd;

    int M{};
    std::cout << "Enter M: "; // Receive input for M
    std::cin >> M;

    int N{};
    std::cout << "Enter N: ";
    std::cin >> N;

    std::vector<int> MContainer;

    // Loop #1: Generate vector of random Integers of M size
    for (int i = 1; i <= M; ++i) {
        srand(rd()); // O(1)
        int randomNum = rand() % 11; // Number between 0-10, O(1)
        MContainer.push_back(randomNum); // Amortized constant: O(1) assumed
    } // O(M)

    // Loop #2: Generate N random integers and count
    std::cout << "N: ";
    int counter{};
    for (int i = 1; i <= N; ++i) {
        srand(rd()); // O(1)
        int randomNum = rand() % 11; // Number between 0-10, O(1)
        std::cout << randomNum << ", "; // 1
        for (auto it = MContainer.begin(); it != MContainer.end(); ++it) {
            if (*it == randomNum) counter++; // 1
        } // O(M)
    } // O(N)

    std::cout << "\n\nCounter: ";
    std::cout << counter;

    return 0;
};
```

Vi starter med at tage input på den ønskede størrelse af M og N . Herefter bruger vi `random_device` fra `<random>`-biblioteket til at seede `rand()` fra `<cstdlib>` i hver iteration af et for-loop, som genererer M tilfældige heltal mellem 0 og 10, der gemmes i en vektor.

Herefter genererer vi en ny runde af N tilfældige heltal og i samme for-loop sammenligner vi det nye tal med alle tal i vektoren og tæller matches. På den måde sparer vi lidt på plads og tid ved ikke at gemme N -værdierne.

Vi valgte at tælle alle matches af M og N for at få det fulde billede, i stedet for at stoppe sammenligningen efter fund af det første match.

- (b) (Week 2) Estimate the worst-case complexity of your program and justify with a line-by-line explanation of the number of steps taken in the program.

Solution:

- Loop #1 har en kompleksitet på $O(M)$, da loopet kun kører en enkelt gang. Vektor-funktionen `push_back` analyseres som en *amortized constant* ifølge dokumentationen [1], hvilket betyder, at den i de fleste tilfælde (bortset fra reallokation) har en kompleksitet på $O(1)$ og kan ignoreres her.
- Loop #2 har et nested loop med størrelse M i det yderste loop med størrelse N . Komplexiteten af dette bliver derfor $O(M \times N)$.

Da $O(M \times N)$ er det største term af de to fundne kompleksiteter, afgør vi, at dette er programmets kompleksitet.

2. (Old exam question, week 2) What is the time complexity (Big-O) of `myMethod`? Argue for your answer.

Solution:

```
int myMethod(int N) {
    int x = 0;
    int y = 0;
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            for (int k = 0; k < N * sqrt(N); k++) {
                x++;
            } // N*sqrt(N)
        } // N*(N*sqrt(N))
    } // N*(N*(N*sqrt(N)))
    for (int i = 0; i < N * N; i++) {
        y++;
    } // N*N

    return x + y;
}
```

Vi analyserer de to loops i `myMethod` som to forskellige kompleksiteter. Det første loop består af 3 nested loops, hvoraf det indre har en øvre grænse på $N \cdot \sqrt{N}$ og de andre på N . Derfor får vi en samlet kompleksitet på:

$$O(N(N(N \cdot \sqrt{N}))) + O(N^2) = O(N^3 \cdot \sqrt{N}) = O(N^{3.5})$$

Det andet loop består af et enkelt loop med en øvre grænse på $N \cdot N$, altså en kompleksitet på $O(N^2)$. Da dette er lavere end $O(N^{3.5})$, konkluderer vi, at tidskompleksiteten bliver $O(N^{3.5})$.

3. (Week 2) For each of the following four program fragments:

- Give an analysis of the running time in Big-O notation.
- Implement the code inside a C function, and give the running time for several values of N .
- Compare your analysis with the actual running times.
- Does compiling/running programs with the `-O` optimization flag change anything in your analysis?

Solution:

For runtime-test kodeimplementering (b), se Bilag A.1.

```
(1) sum = 0; // 1
    for (i = 0; i < n; ++i) {
        ++sum; // 1
    } // 1*n
```

- (a) $1 \cdot n + 1 \Rightarrow O(n)$

Det enkelte loop kører n gange, og hver iteration har en kompleksitet på $O(1)$. Når $n \rightarrow \infty$, dominerer n -leddet den konstante $+1$.

- (b) Runtime-test med `-O0` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	1	0	1	1	2	5	10	16	33	78

- (c) Stemmer overens med $O(n)$: runtime fordobles nogenlunde, når N fordobles ($16 \rightarrow 33 \rightarrow 78 \mu\text{s}$). Værdierne under $N = 2000$ viser 0 eller 1 μs pga. målingsunøjagtighed/støj.

- (d) Runtime-test med `-O1` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	0	0	0	0	1	1	3	7	14	28

Nej, stadig $O(n)$, da `-O1` blot giver en konstant hurtigere eksekvering pga. grundlæggende compileroptimeringer, ikke en ændring i vækstrate.

```
(2) sum = 0; // 1
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n; ++j) {
            ++sum; // 1
        } // 1*n
    } // (1*n)n
```

- (a) $(1 \cdot n)n + 1 = n^2 + 1 \Rightarrow O(n^2)$

Det indre loop kører n gange for hver af de n ydre iterationer, hvilket giver $n \cdot n = n^2$ iterationer i alt. Når $n \rightarrow \infty$, dominerer n^2 -leddet den konstante $+1$.

- (b) Runtime-test med `-O0` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	40	124	564	1525	6708	23562	85029	333914	1331616	5352049

- (c) Stemmer overens med $O(n^2)$: en fordobling af N firedobler runtime ($\sim 4x$).

- (d) Runtime-test med `-O1` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	15	57	226	883	3520	14133	56458	223270	897255	3574629

Nej, stadig hurtigere end `-O0`, men forholdet er fortsat $\sim 4x$, hvilket betyder, at kompleksiteten stadig er $O(n^2)$.

```
(3) sum = 0; // 1
    for (i = 0; i < n; ++i) {
        for (j = 0; j < n * n; ++j) {
            ++sum; // 1
        } // 1*(n*n)
    } // (1*(n*n))n
```

- (a) $(1 \cdot (n \cdot n))n + 1 = n^3 + 1 \Rightarrow O(n^3)$
 Det indre loop kører n^2 gange for hver af de n ydre iterationer, hvilket giver $n \cdot n^2 = n^3$ iterationer i alt. Når $n \rightarrow \infty$, dominerer n^3 -leddet den konstante $+1$.

- (b) Runtime-test med `-O0` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	5244	41689	331261	2667543	21617609	×	×	×	×	×

- (c) Stemmer overens med $O(n^3)$: en fordobling af N giver nogenlunde en 8-dobling af runtime, som forventet for en kubisk algoritme. Det er også grunden til, at vi har sprunget over $N \geq 8000$ da målingerne for disse ville tage flere minutter.

- (d) Runtime-test med `-O1` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	3376	27392	219942	1763053	14007772	×	×	×	×	×

Hurtigere generelt, men forholdet forbliver $\sim 8x$: stadig $O(n^3)$, da loopet ikke blev optimeret væk.

```
(4) sum = 0; // 1
    for (i = 0; i < n; ++i) {
        for (j = 0; j < i; ++j) {
            ++sum; // 1
        } // Loop count changes as 'i' increments,
    } // see analysis below
```

- (a) $\frac{n(n-1)}{2} + 1 = \frac{n^2 - n}{2} + 1 \Rightarrow O(n^2)$
 Det indre loop kører i gange, så det samlede antal iterationer er $0 + 1 + 2 + \dots + (n-1)$, hvilket ifølge Gauss' sum giver $\frac{n(n-1)}{2}$. Når $n \rightarrow \infty$ påvirker faktoren $\frac{1}{2}$ og lavere-ordens led ($-n$, $+1$) ikke vækstraten; kun n^2 gør.

- (b) Runtime-test med `-O0` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	25	78	207	676	2475	9996	42427	165842	660902	2656758

- (c) Stemmer overens med $O(n^2)$: en fordobling af N nærmer sig en $\sim 4x$ forøgelse af runtime, efterhånden som N vokser.

- (d) Runtime-test med `-O1` flag:

N	250	500	1000	2000	4000	8000	16000	32000	64000	128000
μs	8	30	114	444	1764	7280	28944	111500	440955	1797782

Samme forhold på $\sim 4x$: stadig $O(n^2)$, uændret vækstrate.

4. 1. (Old exam question, week 1) Write an implementation of the **Abstract Data Type** **MaxHeap** described below. The implementation **must** be done by using the standard class **vector** from C++ **and it does not have to be efficient**. If you need to iterate through the **vector**, you **must** use **begin()**, **end()**, **++** and *****.

Solution:

```
// VectorHeap.h
#include <vector>
#include "MaxHeap.h"

class VectorHeap : public MaxHeap {
public:
    // is the heap empty?
    bool isEmpty() const override;
    // number of elements in the heap
    int size() override;
    // add an element to the heap
    void insert(const int x) override;
    // find the maximum element in the heap
    const int findMax() const override;
    // delete and return the maximum element of the heap
    int deleteMax() override;

private:
    std::vector<int> _heap;
};

// VectorHeap.cpp
#include <vector>
#include <stdexcept>
#include "VectorHeap.h"

bool VectorHeap::isEmpty() const {
    return _heap.empty(); // 0(1)
}

int VectorHeap::size() {
    return _heap.size(); // 0(1)
}

void VectorHeap::insert(const int x) {
    _heap.push_back(x); // Amortized constant: 0(1) assumed
}

const int VectorHeap::findMax() const {
    if (_heap.empty())
        throw std::out_of_range("findMax() on empty heap"); // 0(1)
    int max = *_heap.begin(); // 0(1)
    for (auto it = _heap.begin(); it != _heap.end(); it++) {
        max = (*it > max) ? *it : max;
    } // 0(n)
    return max;
}

int VectorHeap::deleteMax() {
    if (_heap.empty())
        throw std::out_of_range("deleteMax() on empty heap"); // 0(1)
    int max = *_heap.begin(); // 0(1)
    auto it_max = _heap.begin(); // 0(1)
    for (auto it = _heap.begin(); it != _heap.end(); it++) {
        if (*it > max) {
            max = *it;
            it_max = it;
        } // 0(1)
    } // 0(n)
    _heap.erase(it_max); // 0(n)
    return max;
}
```

Vi implementerer en child-klasse af **MaxHeap** ved navn **VectorHeap**. I de første tre metoder bruger vi metoderne fra Standard Library til at implementere **VectorHeap**. I de sidste to funktioner implementerer vi metoderne manuelt for at kunne bruge iteratorerne til at løope gennem vores vektor. For at kunne returnere **max** gemmer vi både iteratoren og **max**-værdien, når vi looper gennem vektoren. Da **findMax()** og **deleteMax()** ikke er definerede på en tom heap, kaster begge metoder en **std::out_of_range**-exception, hvis heapen er tom.

2. (Week 2) What is the time complexity of the five operations of **MaxHeap**? You must argue for your answers.

Solution:

- **VectorHeap::isEmpty()**
`empty()` har en kompleksitet på $O(1)$ [1].
- **VectorHeap::size()**
Kompleksiteten af `size()` fra 'vector'-klassen er konstant, dvs. $O(1)$ [1].
- **VectorHeap::insert()**
Kompleksiteten af `push_back()` fra 'vector'-klassen er konstant, men amortized. Dvs. den som udgangspunkt er konstant, men kan blive lineær ved omallokering. [1].
- **VectorHeap::findMax()**
Da funktionen looper gennem vektoren en enkelt gang, er der en kompleksitet på $O(N)$, hvor N her er størrelsen på vektoren. Både `begin()` og `end()` er konstante og tilføjer derfor ikke kompleksitet. [1].
- **VectorHeap::deleteMax()**
Løkken gennem vektoren er $O(N)$ ligesom i `findMax()`. Derudover er `erase()` lineær i antallet af elementer efter det slettede [1]. Samlet giver det $O(N)$.

5. 1. Create and implement a C++ class called Library that manages books.

Solution:

```
// Library.h

class Library
{
private:
    std::array<std::list<Book>, 16> categories;
    int totalBook = 0;

public:
    void addBook(int id, int category);
    void displayAllBooks();
    void displayAvailableBooks();
    void displayBorrowedBooks();
    bool borrowBook(int id);
    bool returnBook(int id);
    int countBooksInCategory(int category) const;
};

// Book.h

class Book
{
private:
    int id;
    int category;
    bool borrow;

public:
    Book(int id, int category) : id(id), category(category), borrow(
false){}

    int getId() {
        return id;
    }

    bool getBorrowStatus() {
        return borrow;
    }

    void borrowBook() {
        borrow = true;
    }

    void returnBook() {
        borrow = false;
    }
};
```



```

// Library.cpp

void Library::addBook(int id, int category)
{
    if (category < 0 || category >= static_cast<int>(categories.size()))
        return;

    categories[category].emplace_back(id, category);
    totalBook++;

    int validCategories = 0;
    for (const auto& categoryBooks : categories){
        if (!categoryBooks.empty())
            validCategories++;
    }

    double averageBooks = static_cast<double>(totalBook) / validCategories;
    for (int currentCategory = 0; currentCategory < static_cast<int>(categories.size()); currentCategory++){
        if (!categories[currentCategory].empty() &&
            categories[currentCategory].size() > 2 * averageBooks)
            std::cout << "Warning: category " << currentCategory
                << " contains more than twice the average number of books.\n";
    }
}

// (Note: Vi har valgt at warningen med average gange 2, kun skulle tjekkes ift. aktive kategorier.)

void Library::displayAllBooks()
{
    for (int category = 0; category < 16; category++){
        for (auto& book : categories[category]){
            std::cout << book.getId() << " in category: " << category << std::endl;
        }
    }
}

void Library::displayAvailableBooks()
{
    for (int category = 0; category < 16; category++){
        for (auto& book : categories[category]){
            if (!book.getBorrowStatus())
                std::cout << "\nAvailable books in the library:\n" << book.getId() << " in category: " << category << std::endl;
        }
    }
}

void Library::displayBorrowedBooks()
{
    for (int category = 0; category < 16; category++){
        for (auto& book : categories[category]){
            if (book.getBorrowStatus())
                std::cout << "\nBorrowed books in the library:\n" << book.getId() << " in category: " << category << std::endl;
        }
    }
}

bool Library::borrowBook(int id)
{
    for (auto& category : categories){
        for (auto& book : category){
            if (book.getId() == id && !book.getBorrowStatus())
            {
                book.borrowBook();
                return true;
            }
        }
    }
    return false;
}

bool Library::returnBook(int id)
{
    for (auto& category : categories){
        for (auto& book : category){
            if (book.getId() == id && book.getBorrowStatus())
            {
                book.returnBook();
                return true;
            }
        }
    }
    return false;
}

int Library::countBooksInCategory(int category) const
{
    if (category < 0 || category >= static_cast<int>(categories.size()))
        return 0;

    return static_cast<int>(categories[category].size());
}

```

2. Justify the choice of the data structures you chose to maintain the books in the library.

Solution: Vi har valgt at danne kategorierne som et array på 16 pladser. Dette er fordi der er et fast antal kategorier, og at accesse en position koster $O(1)$. Derefter har vi valgt at lave hver array element som en singly linked list, hvor bøgernes id'er kan indsættes. Grunden til at lists her er valgt, er fordi det kun koster $O(1)$ at indsætte bøger, samt at dette skal fungere dynamisk (ingen fastsat upper bound). Derefter koster det i worst case $O(n)$ at finde items i både arrays og lists, og dermed tabes der ikke meget i forskellen her.

3. Provide the complexity of each operation in the library, and justify.

Solution:

- **Library::addBook()**
`emplace_back()` fra `list`-klassen har en konstant kompleksitet, dvs. $O(1)$ [2]. De to loops kører igennem de 16 kategorier. Da antallet af kategorier altid er 16 og derfor er konstant, har funktionen en samlet kompleksitet på $O(1)$.
- **Library::displayAllBooks()**
Funktionen går igennem alle kategorier og alle bøger i hver kategori. Da alle bøger gennemgås én gang, har funktionen en kompleksitet på $O(N)$, hvor N er det samlede antal bøger.
- **Library::displayAvailableBooks()**
Funktionen går igennem alle kategorier og alle bøger i hver kategori. Hver bog bliver derfor gennemgået én gang. Funktionen har en samlet kompleksitet på $O(N)$, hvor N er det samlede antal bøger.
- **Library::displayBorrowedBooks()**
Funktionen fungerer på samme måde som `displayAvailableBooks()` og går igennem alle bøger én gang. Derfor har funktionen en kompleksitet på $O(N)$.
- **Library::borrowBook()**
Funktionen søger igennem bøgerne, indtil den finder bogen med det ønskede ID. I værste tilfælde findes bogen til sidst eller slet ikke, og alle bøger skal derfor gennemgås. Da gennemgang af en `list` er lineær [2], er kompleksiteten $O(N)$.
- **Library::returnBook()**
Funktionen fungerer på samme måde som `borrowBook()`. I værste tilfælde skal alle bøger gennemgås, og kompleksiteten er derfor $O(N)$ [2].
- **Library::countBooksInCategory()**
Funktionen tilgår direkte den ønskede kategori. `size()` returnerer derefter antallet af bøger i kategorien i konstant tid. Funktionen har derfor en samlet kompleksitet på $O(1)$.

Litteratur

- [1] cplusplus.com. “std::vector,” hentet 3. sep. 2026. webadr.: <https://cplusplus.com/reference/vector/vector/>
- [2] cplusplus.com. “std::list,” hentet 5. sep. 2026. webadr.: <https://cplusplus.com/reference/list/list/>

A Bilag

A.1 Exercise #3 - main.cpp

```

1  #include <vector>
2  #include <iostream>
3  #include <chrono>
4  #include <functional>
5
6  void frag1(int n) {
7      int sum = 0;
8      for (int i = 0; i < n; ++i) {
9          ++sum;
10     } // O(N)
11 }
12 void frag2(int n) {
13     int sum = 0;
14     for (int i = 0; i < n; ++i) {
15         for (int j = 0; j < n; ++j) {
16             ++sum;
17         } // O(N)
18     } // O(N^2)
19 }
20 void frag3(int n) {
21     int sum = 0;
22     for (int i = 0; i < n; ++i) {
23         for (int j = 0; j < n * n; ++j) {
24             ++sum;
25         } // O(N^2)
26     } // O(N^3)
27 }
28 void frag4(int n) {
29     int sum = 0;
30     for (int i = 0; i < n; ++i) {
31         for (int j = 0; j < i; ++j) {
32             ++sum;
33         }
34     }
35 }
36
37 void runtime_test(
38     const std::string& test_name,
39     const std::function<void(int)>& operation,
40     const std::vector<int>& N_values
41 ) {
42     std::cout << "----- Runtime test of " << test_name << " ----- \n";
43     for (auto n_value : N_values) {
44         auto t_start = std::chrono::steady_clock::now();
45         operation(n_value);
46         auto t_end = std::chrono::steady_clock::now();
47
48         auto t_diff_us = std::chrono::duration_cast<std::chrono::microseconds>(t_end - t_start);
49         auto t_diff_ms = std::chrono::duration_cast<std::chrono::milliseconds>(t_end - t_start);
50         std::cout << "N = " << n_value << "\n";
51         std::cout << "Elapsed: " << t_diff_us.count() << " us / " << t_diff_ms.count() << " ms \n\n";
52     }
53     std::cout << "\n\n";
54 }
55
56 int main() {
57     std::cout << "\n\n\n";
58     runtime_test("Fragment #1", frag1, {250, 500, 1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000});
59     runtime_test("Fragment #2", frag2, {250, 500, 1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000});
60     runtime_test("Fragment #3", frag3, {250, 500, 1000, 2000, 4000}); // Fragment #3 is too slow for
61     runtime_test("Fragment #4", frag4, {250, 500, 1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000});
62     return 0;
63 }

```